Numpy: It is python library that stands for numeric python

```
It provides support fir large, ,ulti dimensional arrays and matrics, along with collection of mathematical function to operates on these array efficiently
```

Need:

```
Array: It introduce concept of arrays which are similar to lists in python but store and manipulates large amount of data more efficiently

Numerical operation: Logarithms, exponentials

Broadcasting: Allows operations between array of different shape and size

Efficiency: It is faster than python list

Integration with libraries: scipy, pandas,matplotlib
```

Numpy vs List

```
Numpy is fix
List is dynamic

Numpy contains homogenis data
List contains all types of data

Numpy perform difficult operation easily
List required large coding
```

```
import numpy as np
arr1=np.array([1,2,3,4,5])
print(arr1)
print(type(arr1))
```

```
[1 2 3 4 5]
<class 'numpy.ndarray'>
```

```
'''creating 2d array(matrix)
    collection of 1d array
    Matrices are sppecially designed for linear algebra operation
    such as matrix multiplication, inversion and solving linear equations
'''
matrix=np.array([[1,2,3],[4,5,6]])
print(matrix)
print(type(matrix))
```

```
[[1 2 3]
 [4 5 6]]
<class 'numpy.ndarray'>

''' creating 3d array(tensor)
    collection of 2d array
    Commonly used in deep learning framework for representating
    and manipulating multi dimensional data, such as images, vides
    and sensor data
'''
tensor=np.array([[[1,2,3],[4,5,6]],[[7,8,9],[10,11,12]]])
print(tensor)
print(type(tensor))

[[[ 1  2  3]
  [ 4  5  6]]

 [[ 7  8  9]
  [10 11 12]]]
<class 'numpy.ndarray'>

# compare list and numpy

import numpy as np
import time
n=100000
np_array=np.arange(n)
py_list=list(range(n))
start_time=time.time()
np_sum=np.sum(np_array)
numpy_time=time.time()-start_time
print(f"numpy time {numpy_time}")
print(np_sum)
start_time=time.time()
py_sum=sum(py_list)
python_time=time.time()-start_time
print(f"python time {python_time}")
print(py_sum)


numpy time 0.0009634494781494141
704982704
python time 0.0027320384979248047
4999950000

a=[i for i in range(10000)]
import sys
import numpy as np
print(sys.getsizeof(a))
b=np.arange(10000)
print(sys.getsizeof(b))
```

```
85176
40112
```

```
r1=np.array([[1,2,3],[4,5,6]])
print(r1.shape) #output 2 rows and 3 columns
print(r1.size)#it return number of elements in it
print(r1.dtype) #Data type of array
print(r1.ndim) #Number of dimention in array
```

```
(2, 3)
6
int32
2
```

```
'''Mean: The mean is the average of all the numbers in a dataset.
To calculate the mean, you add up all the numbers and then divide by
the total number of values. The mean is sensitive to extreme values
(outliers)
in the dataset, as it takes into account the magnitude of each value.

Median: The median is the middle value in a dataset when the values
are arranged
 in ascending or descending order. If the dataset has an odd number of
values,
   the median is the middle number. If the dataset has an even number
of values,
     the median is the average of the two middle numbers. The median
is not
     influenced by extreme values (outliers) and is often used when
the dataset
       contains such values.'''

r2=np.array([12,44,10,55,23,47,88,74,32])
print(f"Mean: {np.mean(r2)}")
print(f"Median {np.median(r2)}")
print(f"Standard Deviation {np.std(r2)}")
```

```
Mean: 42.77777777777778
Median 44.0
Standard Deviation 25.204839825265385
```

```
#reshape the array in new shape
r3=np.arange(12)
reshape_array=r3.reshape(3,4)
print(reshape_array)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

```
#generate an array of random numbers using numpy
r4=np.random.rand(3,2)# it will generate 3*2 matrix with number
between 0 to 1
print(r4)
```

```
[[0.92013554 0.34716806]
 [0.63341145 0.4517245 ]
 [0.61663877 0.36407495]]
```

```
#transpose of matrix
r5=np.array([[1,2,3],[4,5,6]])
tran_matrix=np.transpose(r5)
print(r5)
print(tran_matrix)
```

```
[[1 2 3]
 [4 5 6]]
[[1 4]
 [2 5]
 [3 6]]
```

```
'''np.vstack(): This function is used to stack arrays vertically,
i.e., it stacks arrays row-wise. It takes a sequence of arrays and
stacks them vertically
to create a new array. '''
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.vstack((a, b))
print(result)
# Output:
# [[1 2 3]
#  [4 5 6]]
```

```
[[1 2 3]
 [4 5 6]]
```

```
'''np.hstack(): This function is used to stack arrays horizontally,
i.e., it stacks arrays column-wise. It takes a sequence of arrays and
stacks them horizontally to create a new array. '''
import numpy as np
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
result = np.hstack((a, b))
print(result)
# Output:
# [1 2 3 4 5 6]
```

```
[1 2 3 4 5 6]
```

```
''' When np.where() is called with a single argument (the condition),
it returns a tuple of arrays, one for each dimension, containing the
indices
where the condition is true.'''
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
indices = np.where(arr > 2)
print(indices)
# Output: (array([2, 3, 4]),)
```

```
(array([2, 3, 4], dtype=int64),)
```

```
'''Using with Two Arguments: When np.where() is called with two
arguments,
it returns the indices where the condition is true from the first
array, and
the corresponding elements from the second array where the condition
is false.'''
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
new_arr = np.where(arr > 2, arr, 0)
print(new_arr)
# Output: [0 0 3 4 5]
```

```
[0 0 3 4 5]
```

```
'''Using with Three Arguments: When np.where() is called with three
arguments,
 it returns elements from either the second or third argument based on
whether
 the condition is true or false.'''
import numpy as np
arr = np.array([1, 2, 3, 4, 5])
new_arr = np.where(arr > 2, 'yes', 'no')
print(new_arr)
# Output: ['no' 'no' 'yes' 'yes' 'yes']
```

```
['no' 'no' 'yes' 'yes' 'yes']
```